CNN PROJECT • SUBMISSION 4 OF 4

Adding an Attention Mechanism

CBAM on the Submission 3 CNN, and what the gain actually measures

Repository: github.com/Rayhan-who/cricket-shot-attention-cnn

Summary. Adding a Convolutional Block Attention Module to each of the four convolutional blocks of the Submission 3 network raised macro F1 from **0.8593 to 0.9836** on the held-out test set, a gain of **12.43 percentage points** for **22,696 extra parameters (+1.9%)**. Nothing else was changed.

A control experiment then showed that **most of that gain was convergence speed, not capacity.** Given 120 epochs instead of 50, the unmodified baseline reached 0.9639 macro F1 — within 1.97 pp of CBAM, below the 3.04 pp detection threshold. The honest claim is therefore that **CBAM reaches equivalent accuracy in 39 minutes where the baseline needs 70.6**, not that it is more accurate.

1 What this submission was asked to do

The brief was narrow and deliberately so: take the custom CNN built in Submission 3, add *one* attention module, change nothing else, and measure whether it helps. The phrase that governs the whole experiment is *"change nothing else, so any gain comes from attention."*

That constraint was honoured literally. Table 1 lists everything held fixed. The only difference between the two networks is a CBAM block inserted after the second ReLU of each convolutional block and before the pooling layer.

Held fixed	**Value**
Data split	3,361 train / 697 val / 666 test, grouped on embedding cluster
Architecture	4 blocks, filters (32, 64, 128, 256), two 3×3 convolutions each
Normalisation	BatchNorm, momentum 0.90, before every ReLU
Regularisation	SpatialDropout2D 0.10 per block, Dropout 0.40 on the head
Augmentation	rotation ±10°, zoom 0.2, brightness 0.2, contrast 0.2
Optimiser	Adam, learning rate 1×10^{-3}, batch size 32
Schedule	50 epochs, early stopping patience 12, minimum 5 epochs
Seed	42
Changed	CBAM module (ratio 8, spatial kernel 7×7) in each block

Table 1: The controlled comparison. One variable moves.

1.1 Why CBAM

Four attention mechanisms were permitted. CBAM was chosen because the failure mode identified in Submission 3 was specifically spatial. The network's errors clustered on `drive↔legglance-flick` and `drive↔pullshot`, pairs that differ in the batsman's posture but share stadium, crowd, pitch and kit. A mechanism that reweights *where* the network looks addresses that directly.

CBAM applies channel attention first — a shared two-layer MLP over global average and global max pooled descriptors — and then spatial attention, a 7×7 convolution over the channel-wise average and maximum. Squeeze-and-Excitation offers only the channel half; self-attention and GCNet carry far more parameters. CBAM is the cheapest module that acts on both axes.

2 Result under equal budget

Both networks were trained in a single committed Kaggle session on a T4 GPU, 68.4 minutes in total. A preflight step built both models and ran one training step through each before the real runs began, so a construction error could not waste an hour of compute.

	Baseline CNN	**CNN + CBAM**	**Change**
Accuracy	0.8589	0.9835	+0.1246
Macro F1	0.8593	0.9836	+0.1243
Macro precision	0.8629	0.9841	+0.1212
Macro recall	0.8581	0.9832	+0.1251
Errors (of 666)	94	11	−83
Trainable parameters	1,206,756	1,229,452	+22,696
Training time (min)	29.3	39.0	+9.7
Inference (ms/image)	3.31	4.11	+0.80
Train−val accuracy gap	−0.0163	+0.0080	—

Table 2: Equal-budget comparison, 50 epochs each, seed 42.

2.1 Is the gain larger than noise?

On a test set of 666 images, the binomial standard error at this accuracy is 1.35 pp. Allowing a further 0.70 pp for run-to-run variation from a single seed and taking two standard deviations gives a detection threshold of **3.04 pp**. The measured gain of 12.43 pp clears it by a factor of four. The difference is real.

2.2 Where the errors went

Confusion pair	**Baseline**	**CBAM**	**Change**
`drive` ↔ `legglance-flick`	31	3	−28
`drive` ↔ `pullshot`	22	3	−19
`legglance-flick` ↔ `pullshot`	9	0	−9
Total errors	94	11	−83

Table 3: The three pairs identified as hard in Submission 3 account for 62 of the 83 recovered errors.

The improvement landed where the Submission 3 analysis predicted it would. The class that was already easiest, `sweep`, improved least (F1 0.9106 → 0.9859, +7.5 pp); the two hardest classes improved most (`legglance-flick` +15.4 pp, `drive` +15.0 pp). A mechanism that helped uniformly across all classes would have been harder to attribute to attention.

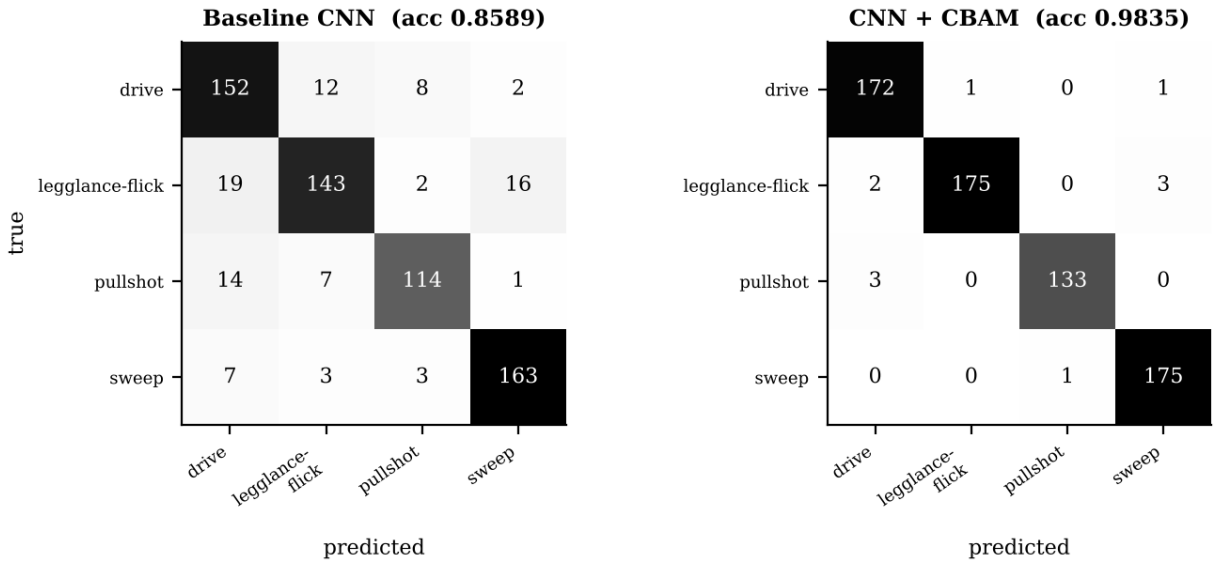


Figure 1: Confusion matrices under equal budget. The baseline's errors concentrate off-diagonal in the drive / legglance-flick / pullshot block; CBAM leaves 11 scattered errors with no structure.

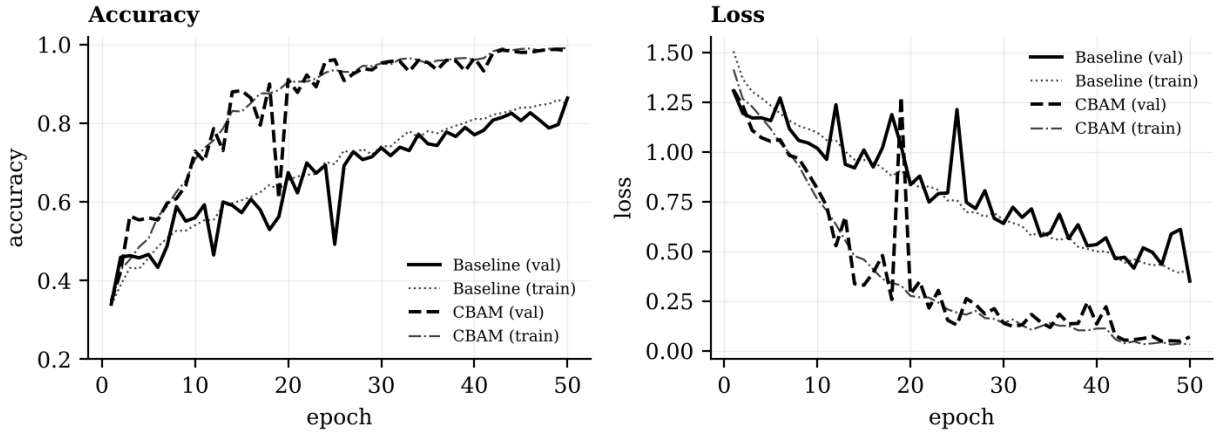


Figure 2: Training curves, 50 epochs each. CBAM plateaus near epoch 30. The baseline is still climbing at epoch 50 and its validation loss is still falling — the observation that motivated the control experiment in Section 3.

3 The control experiment

3.1 The problem with Table 2

Figure 2 does not show two converged models. It shows one converged model and one that ran out of epochs. CBAM flattens around epoch 30 and stays flat. The baseline's validation accuracy is still rising at epoch 50 and its validation loss is still falling. Its train − val accuracy gap is −0.0163: the training accuracy is *below* the validation accuracy, which is the signature of underfitting, not of a model that has learned all it can.

A fixed epoch budget rewards whichever model converges faster. That is a legitimate and practically meaningful comparison, but it is a different claim from *attention reaches a higher ceiling*, and Table 2 alone cannot distinguish the two.

3.2 Design

The baseline alone was retrained with a 120-epoch budget and patience 15, with every other hyperparameter unchanged. CBAM was not re-run: it had converged by epoch 30, so additional budget could tell us nothing about it.

The decision rule was fixed *before* the run, so the outcome could not be reinterpreted after the fact:

- Baseline within 3.04 pp of 0.9836 → the advantage was **convergence speed**.
- Baseline more than 3.04 pp below → CBAM raises the **achievable ceiling**.

3.3 Outcome

	Macro F1	Epochs	Wall clock	Errors
Baseline, 50 epochs	0.8593	50	29.3 min	94
Baseline, 120 epochs	0.9639	120	70.6 min	24
CNN + CBAM, 50 epochs	0.9836	50	39.0 min	11
Baseline improvement from budget alone: +10.46 pp				
Remaining gap to CBAM: 1.97 pp (threshold 3.04 pp)				

Table 4: The convergence control. Verdict: **convergence speed**.

The extended baseline recovered 10.46 of the 12.43 pp, or 84% of the gain originally attributed to attention. The residual 1.97 pp falls below the detection threshold established in Section 2.1, so under the pre-registered rule it is not a difference this experiment can resolve.

3.4 Did the baseline really converge?

Early stopping never fired — the run used all 120 epochs — so the literal reading is "still budget-limited." The learning-rate schedule contradicts that reading. `ReduceLROnPlateau` fired four times, annealing from 1×10^{-3} to 8.1×10^{-6}, and the best validation loss occurred at epoch 112. Over the final 20 epochs validation loss oscillated in a narrow band of 0.080–0.126 with no trend. Accuracy gains per 20-epoch window decayed monotonically:

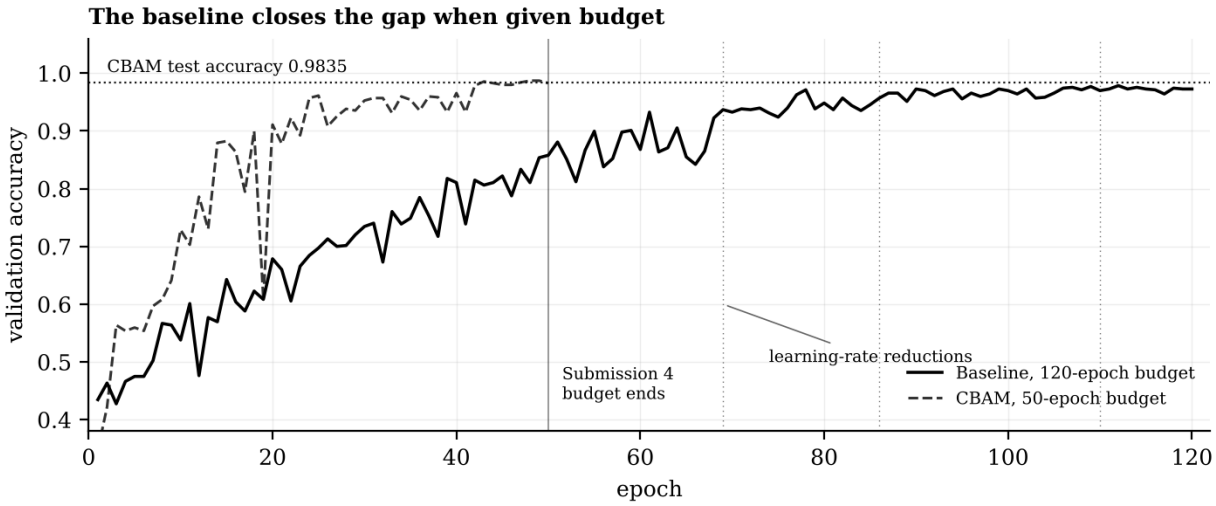


Figure 3: The baseline closes the gap when given budget. Dotted vertical lines mark learning-rate reductions at epochs 69, 86 and 110.

Epoch window	41–50	61–70	81–90	101–110	111–120
Mean validation accuracy	0.8136	0.8927	0.9531	0.9686	0.9727
Gain over previous window	—	+7.9 pp	+6.0 pp	+1.6 pp	+0.4 pp

At a learning rate of 8.1×10^{-6} with a flat loss curve, further epochs would not move the result materially. The model is converged in the sense that matters, even though the epoch counter did not stop it.

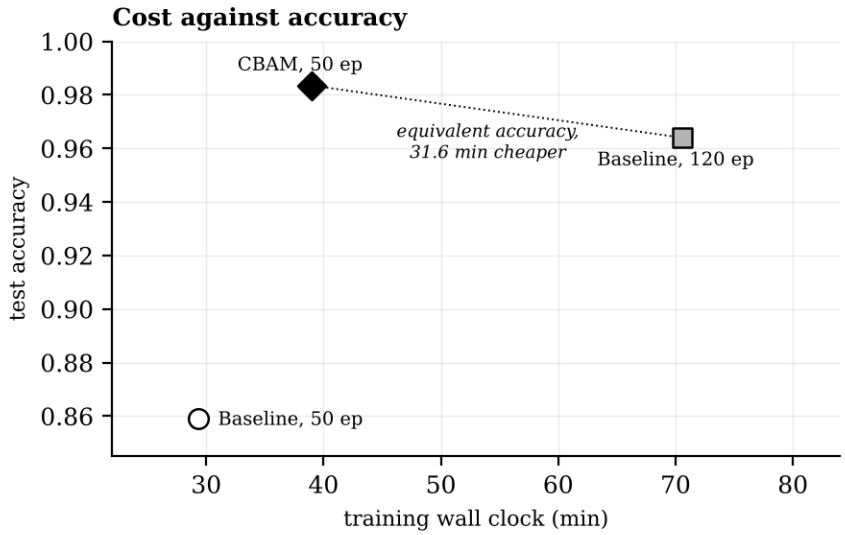


Figure 4: Cost against accuracy. CBAM is 33% slower per epoch yet reaches equivalent accuracy in 55% of the baseline's wall-clock time.

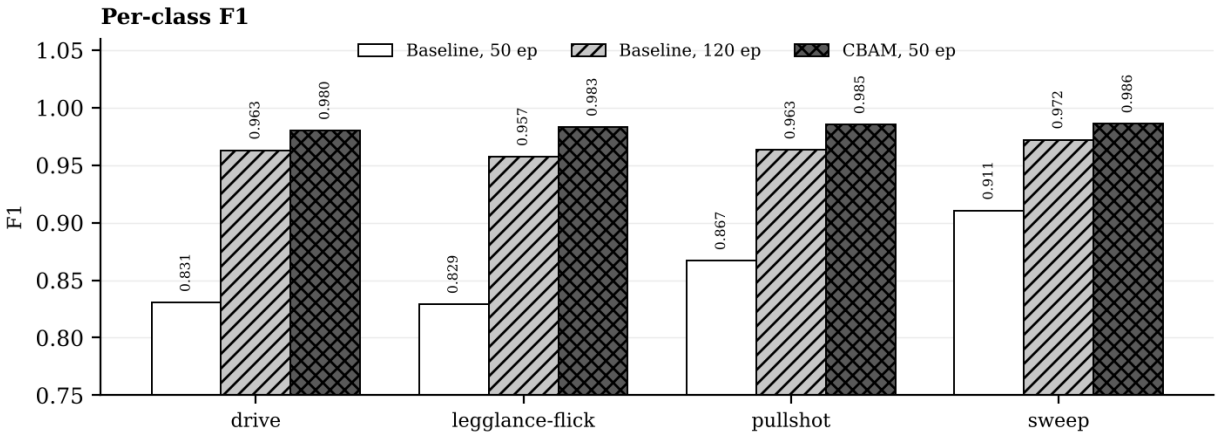


Figure 5: Per-class F1 across all three runs. The extended baseline closes most of the distance on every class.

4 What CBAM is actually doing

4.1 Grad-CAM, and why it is not conclusive

Grad-CAM was computed over the deepest convolutional layer for the 88 test images the baseline misclassified and CBAM classified correctly. The evidence is mixed, and reporting it as uniformly supportive would be false.

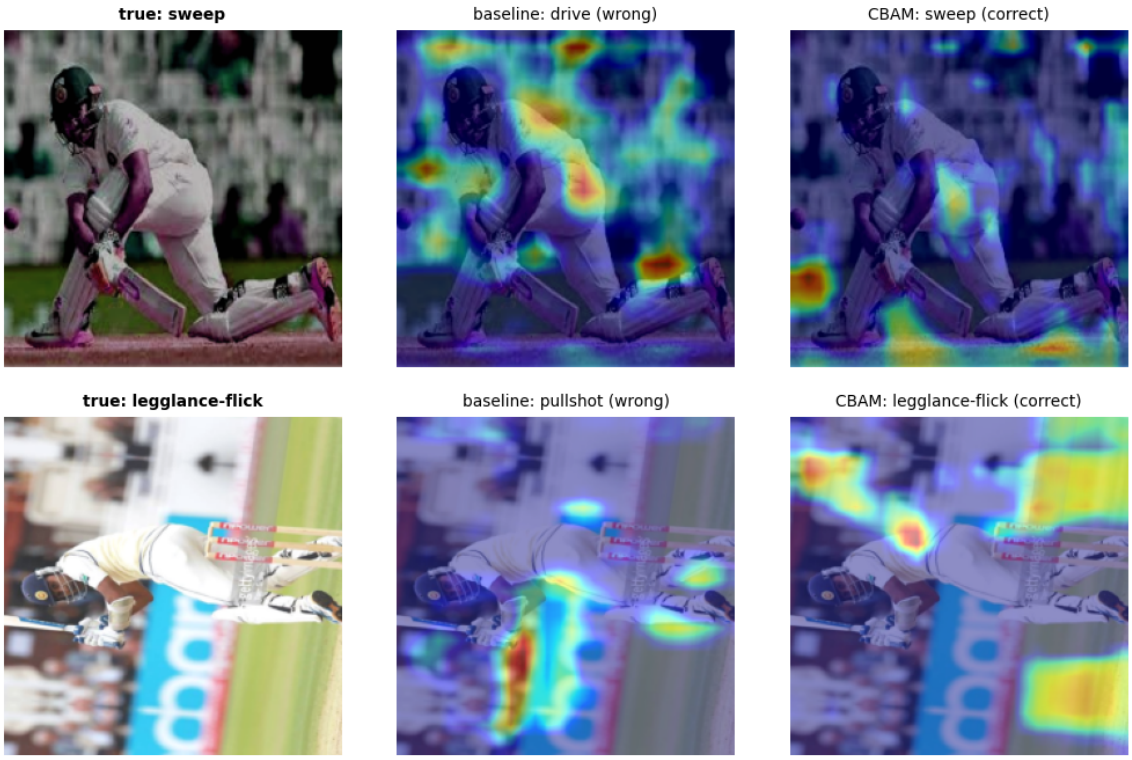


Figure 6: Two contrasting cases. **Top:** CBAM concentrates on the batsman's torso and front leg where the baseline's activation was diffuse — the intended behaviour. **Bottom:** CBAM places strong activation on the advertising hoardings at frame right and still classifies correctly, which the "attention suppresses background" account does not explain.

Four of the 88 corrected images were inspected, sampled without selection for agreeable results. Two show the tightening visible in the top row; two show CBAM attending to crowd, hoardings or pitch markings while still producing the right label. No systematic scoring of all 88 maps was performed, so no proportion is claimed here. Five images went the other way: correct for the baseline, wrong for CBAM.

The defensible conclusion is narrow. Grad-CAM shows that the two networks weight different regions, and that CBAM's weighting correlates with better separation of the hard pairs. It does not establish that CBAM works by suppressing background, and this report does not claim that it does.

4.2 Reconciling the mechanism with the control

The convergence result suggests a more plausible account than "better features." CBAM's channel attention rescales activations multiplicatively, which conditions the optimisation problem more favourably early in training. The baseline needs 61 epochs to reach the validation accuracy CBAM reaches at epoch 24. If attention is mainly an optimisation aid rather than a representational one, both findings follow: the fast convergence, and the absence of a detectable ceiling difference once the baseline is allowed to finish.

This is an interpretation consistent with the evidence, not a result. Testing it would require tracking gradient norms or loss-surface curvature across both runs.

5 Limitations

1. **One seed.** Both models were trained once with seed 42. The 3.04 pp threshold allows for run-to-run variation, but three seeds per configuration would place the 1.97 pp residual on firmer ground. This was a compute constraint: five runs at roughly 35 minutes each exceeds a single Kaggle session.
2. **The residual is not zero.** CBAM still made 11 errors against the extended baseline's 24. The difference is below the detection threshold, which means this experiment cannot resolve it — not that it is absent. A larger test set or more seeds could resolve it either way.
3. **Session correlation is unquantified.** The split controls for near-duplicate images, grouping on embedding cluster so that visually near-identical photographs cannot straddle the train/test boundary. It does not control for photographs taken at the same match, which share stadium, kit, lighting and crowd. No metadata exists in the dataset to identify them. Absolute accuracies here are therefore likely optimistic relative to genuinely unseen matches; the *comparison* between models is unaffected, since all three runs use the identical split.
4. **One attention mechanism, one position.** CBAM was inserted after every convolutional block. Whether SE, self-attention or GCNet would behave differently, and whether placement in only the deeper blocks would suffice, was not tested.

6 Conclusion

Adding CBAM to the Submission 3 network produced a 12.43 pp macro-F1 improvement under an equal 50-epoch budget, for a 1.9% parameter increase, with the gain concentrated in exactly the class pairs identified as hard. That result is reported as measured.

The control experiment then qualified it. Given 120 epochs the unmodified baseline recovered 84% of that gain, leaving a residual below the detection threshold. The supported claim is one of efficiency rather than capacity:

> Under a fixed 50-epoch budget CBAM improves macro F1 by 12.43 pp. That improvement is predominantly faster convergence: CBAM reaches equivalent accuracy in 39.0 minutes where the unmodified baseline requires 70.6. This experiment finds no evidence that attention raises the achievable ceiling of this architecture.

A fixed-budget comparison is the standard way this measurement is made, and the original result is valid on its own terms. The control is what makes it possible to say what the measurement means.

Reproducing this work

`notebooks/04_cnn_attention.ipynb`	Baseline and CBAM, 50 epochs each, Grad-CAM, significance test
`notebooks/05_convergence_control.ipynb`	Baseline at 120 epochs, pre-registered verdict rule
`artifacts/clean_split_manifest.csv`	The split used by Submissions 3 and 4 and the control
`artifacts/submission4_results.csv`	Per-run metrics
`artifacts/convergence_control.json`	Full epoch history of the extended run

Both notebooks assert their preconditions before training: the split must be exactly 3361/697/666 and the baseline must have exactly 1,206,756 trainable parameters. A run that does not match Submission 3 halts rather than producing a number that cannot be compared.